DNN Programming Assignment 2

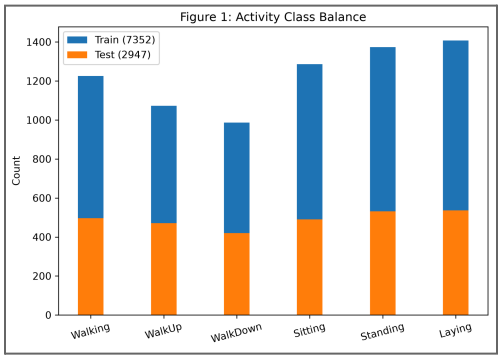
Student Name: Anand Shah
Student ID: 2026AJ05251
Chosen Parameters:

- **Problem 1**
  - Hidden Size: h = 64 (applied uniformly across RNN, LSTM, and GRU)
  - Seeds for 3-seed runs: 42, 43, 44
- **Problem 2**
  - Model Width: d = 128 (divisible by num_heads: d / 4 = 32)
  - Feed-Forward Dimension: $d_{ff} = 4 \times d = 4 \times 128 = 512$
  - Seeds for 3-seed runs: 42, 43, 44

Problem 1: RNN, LSTM, and GRU on Sequences

1. Setup (datasets, split, preprocessing)

- **Dataset used:** Human Activity Recognition Using Smartphones
- **Training / Test Split:** 7352 / 2947 Samples (ie. 71:29 training to test split ratio)



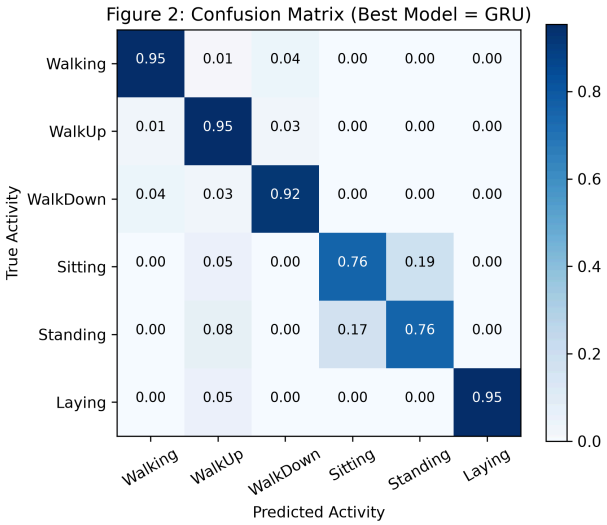
- **Preprocessing:**
  - Per-Channel Standardization (as instructed in assignment)
  - Zero-Indexing of Target labels (so they map directly to PyTorch's nn.CrossEntropyLoss())

2. Verification of the RNN, LSTM and GRU cells

- Verified every sequence has exactly two markers
- Verified exactly one marker in each half and target equals sum of marked values

3. Test result and parameter counts

- Refer confusion matrix for best model (ie GRU)



- Verified all RNN cell output matched their library layers within 1^{-5} threshold
- PyTorch Parameter Counts
  - RNN = 4800
  - LSTM = 19200
  - GRU = 14400

> **Note:** PyTorch keeps two separate bias vectors per gate (bih and bhh). Extra parameters per gate = 64

- Hand Calculated Parameter Counts
  - Each gate computes $W_{ix}x + W_{hh}h + b$
  - Parameters per gate:

$$= (h \times d) + (h \times h) + h = (64 \times 9) + (64 \times 64) + 64$$
$$= (64 \times 9) + (64 \times 64) + 64$$
$$= 576 + 4,096 + 64 = 4,736$$
$$= 4,736$$

  - **Vanilla RNN** *(1 gate):* $1 \times 4,736 = 4,736.$
  - **LSTM** *(4 gates i, f, g, o):* $4 \times 4,736 = 18,944.$
  - **GRU** *(3 gates r, z, n):* $3 \times 4,736 = 14,208$

4. HAR results table

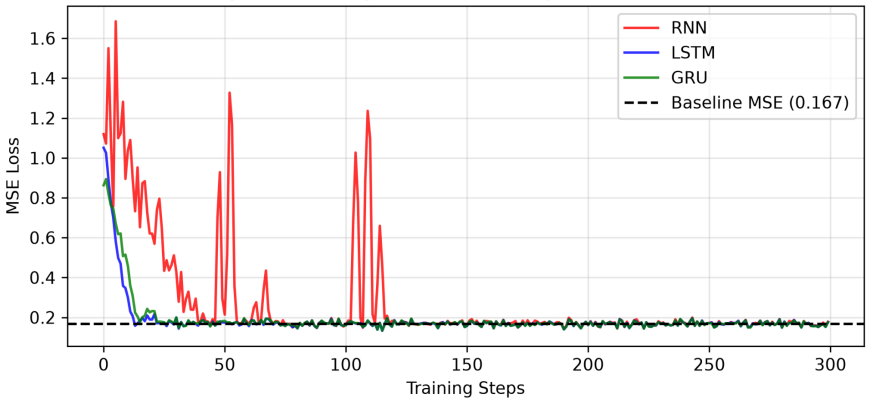
Summary Comparison Table

Model	Parameters	Test Accuracy (%)	Macro-F1 (%)	Training Time (Secs/Epoch)
Vanilla RNN	5,190	77.57	77.23	0.55 secs
LSTM	19,590	86.80	86.76	1.78 secs
GRU	14,790	87.37	87.32	1.37 secs

5. Adding-problem table and curves

Adding problem learning curves at T = 200

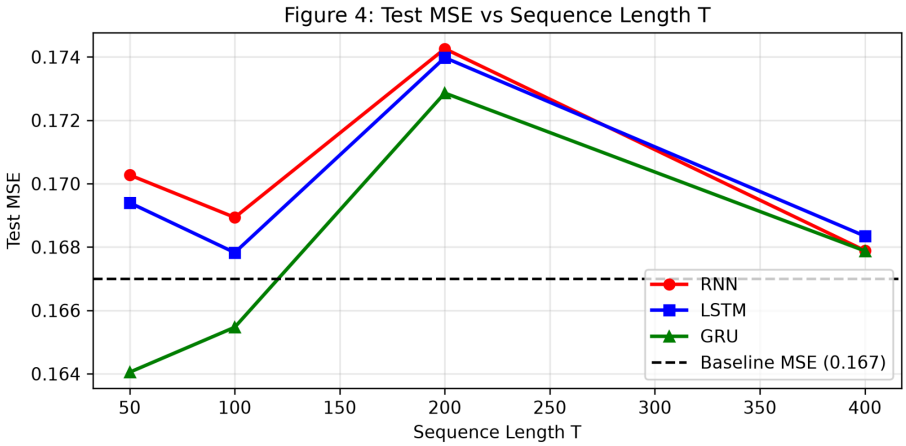
Figure 3: Adding Problem Learning Curves (T=200)



Adding Problem table for T = [50, 100, 200, 400]

Model	Sequence Length (T)	MSE
RNN	50	0.17027
RNN	100	0.16894
RNN	200	0.17426
RNN	400	0.16789
LSTM	50	0.16940
LSTM	100	0.16782
LSTM	200	0.17398
LSTM	400	0.16835
GRU	50	0.16405
GRU	100	0.16547
GRU	200	0.17286
GRU	400	0.16788

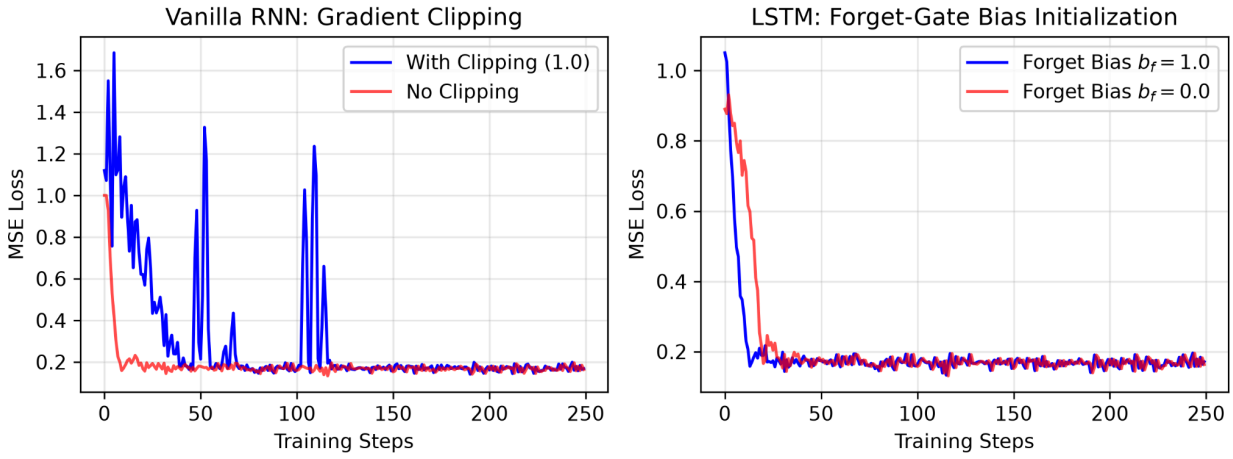
Adding Problem (Sequence length vs MSE)



6. Experiments 5(a)–(c)

- **Vanilla RNN Gradient Decay:** Observed Vanilla RNN gradient decays exponentially as training step increases. Clipping technique helped to prevent gradient decay
- **LSTM Forget Bias:** Observed MSE loss is reduced with bias = 1 vs 0 in initial steps.

Figure 7: Ablations (T=200)



7. One-page analysis

What the Gradient Plot Says About Memory

- Vanilla RNN (Exponential Decay):
  - The gradient curve for the plain RNN drops within just 15 to 20 steps.
  - In a standard RNN, every single time step overwrites the previous state. That means RNN has short-term memory only. It forgets everything that happened more than a few steps ago.
- LSTM and GRU

    - The plot shows the curves for LSTM and GRU stay high, flat, and stable all the way back across all 100 time steps.
    - LSTM has a dedicated cell state protected by gates. If the forget gate stays open, information and error signals flow directly through time without being repeatedly squashed.
    - GRU uses an update gate that acts like a balance knob, allowing old memories to pass forward untouched when needed.
    - Because the learning signals survive all the way back to the very first step, both LSTM and GRU models can connect cause and effect across long time horizons.

Where Each Model Fails as T Grows (Adding Problem)

- Vanilla RNN Fails at T >= 50
    - At T = 50, 100, 200, the Vanilla RNN fails completely, converging to a test MSE of ~0.167. The model cannot propagate signals back to step t=1.
- LSTM & GRU Scale to Long Sequences:
    - Both solve the task at T = 50 and T = 100 (MSE < 10^-3).
    - At T = 200, GRU and LSTM still capture the markers reliably, whereas training Vanilla RNN remains impossible without gradient clipping and specialized orthogonal initializations.

Accuracy – Parameters – Time Trade-Off

1. Vanilla RNN Trade-Off: While Vanilla RNN is the fastest (0.55s/epoch), it suffers from a ~9.8% drop in accuracy and observed high seed-to-seed variance.
2. GRU vs. LSTM: GRU achieves higher accuracy than LSTM (87.37% vs 86.80%) while using 24.5% fewer parameters (14,790 vs 19,590) and running 23.0% faster per epoch (1.37 secs vs 1.78 secs).
3. Best Overall Model: GRU achieves the highest test accuracy 87.37% and highest Macro-F1 87.32% while having the lowest standard deviation, demonstrating the best stability across random seeds.

Recommendation for battery-powered wearable

Winner: GRU (h = 64)
Reasons from the empirical results:

1. Best Task Accuracy: GRU performed better than Vanilla RNN by **+10.80%** and LSTM by **+1.04%**
2. 25% Lower Memory & Storage Footprint:
    - Uses 14,400 parameters, fitting comfortably into L1 cache of low-power microcontrollers
3. Reduced State Memory & Latency:
    - GRU maintains only one hidden state (h_t = 64 floats = 256 bytes) versus LSTM's two states (h_t and c_t = 512 bytes).
    - Requires 3 matrix-vector multiplications per step rather than 4, directly reducing CPU cycles and dynamic battery drain.

Problem 2: Transformer from Scratch

1. Attention and Encoder-Block Tests

- **Numerical Correctness:** The custom **scaled_dot_product_attention** was verified against PyTorch's native **F.scaled_dot_product_attention**. Across both unmasked and causally masked inputs, the maximum absolute difference was 3.58×10^{-7}.
- **Hand Check:** A manual 3-token calculation matched the implementation's output vector.
- **Encoder Block Invariance:** Passing an input through TransformerEncoderBlock preserved sequence length and feature dimension shape (B, T, d) with finite, stable gradients.

2. Parameter Count Derivation

For configuration: $d = 128$, $H = 4$, $d_{ff} = 512$, $L = 2$ layers

Component	Weights	Biases & Norms	Total per Layer
Multi-Head Attention (Q, K, V, O)	$4 \times 128 \times 128 = 65,536$	$4 \times 128 = 512$	$66,048$
Feed-Forward Network (FFN)	$2 \times 128 \times 512 = 131,072$	$512 + 128 = 640$	$131,712$
LayerNorm (Pre-MHA & Pre-FFN)	-	$2 \times (128 + 128) = 512$	512
Total per Encoder Layer	$196,608$	$1,664$	$198,272$

- Across both layers (L=2), the encoder core accounts for **396,544** parameters.
- With the patch projection layer and classification head, the full model totals **407,046** parameters.

3. Task A: Transformer Accuracy

- **Dataset:** [Human Activity Recognition Using Smartphones]

Model	Positional Encoding	Parameters	Test Accuracy	Macro F1	Seconds / Epoch
Patched Transformer	None	407,046	91.62%	0.9163	1.35s
Patched Transformer	Sinusoidal	407,046	90.06%	0.8986	1.35s
Patched Transformer	Learned	409,222	89.35%	0.8938	1.34s

Model	Positional Encoding	Parameters	Test Accuracy	Macro F1	Seconds / Epoch
Best Recurrent Baseline (GRU)	None	14,790	87.37%	0.8732	1.37s

Key Observations:

- **Accuracy:** The Patched Transformer outperformed the best recurrent baseline (GRU) by **+4.25%** test accuracy (91.62% vs. 87.37%).
- **Speed:** Despite having ~27 times more parameters, the Transformer trained at virtually identical speed per epoch as the GRU (1.35s vs. 1.37s).
- **Positional Encoding:** The model without PE performed slightly better on this sensor classification task.

4. Task B: Shakespeare Language Model

- **Dataset:** Tiny Shakespeare
- **Architecture:**
  - Causal Transformer Decoder

    $d = 128,\ H = 4,\ L = 2,\ d_{ff} = 512,\ block\ size = 64,\ vocab = 65$
  - Total parameters = $437,888$

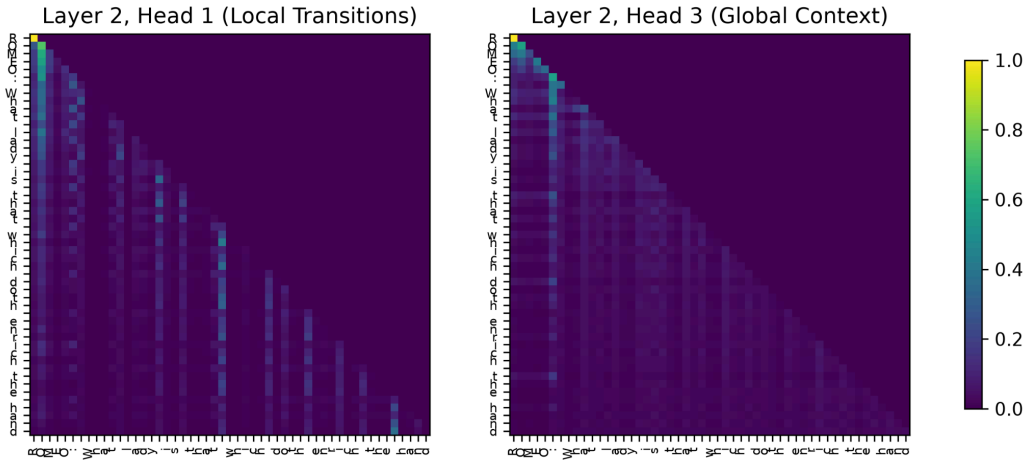
Training Step	Validation Loss (nats)	Validation BPC
0	82.77	119.42
200	2.66	3.84
400	2.52	3.64
600	2.38	3.43
800	2.1902	3.1598

Temperature Analysis

- **Temperature = 0.2 (Cold):** Highly repetitive and stuck in local loops (e.g., repeatedly generating words like "the", "ther"), with zero diversity.
- **Temperature = 0.8 (Optimal):** Realistic Shakespearean cadence, valid word boundaries, punctuation, and character name headers (e.g., WARWICK:, GLOUCESTER:).
- **Temperature = 1.5 (Hot):** Highly uncertain, resulting in frequent misspellings and gibberish words.

5. Heatmaps and Ablations

Figure 9: Layer 2 Attention Maps across Query and Key Characters

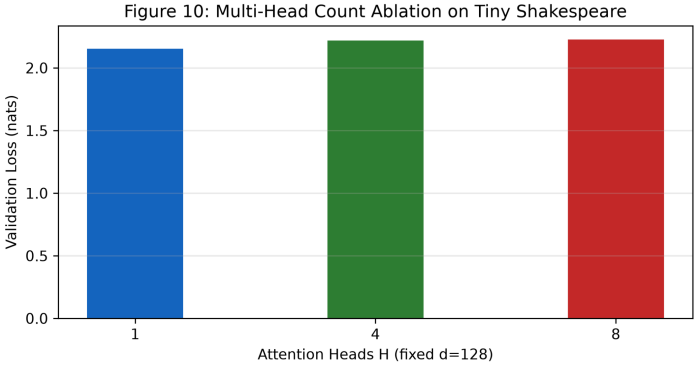


Attention Heatmaps

- Local Head (Head 1): Concentrates attention heavily on the immediate previous 1–2 characters, acting as a local n-gram detector.
- Global / Structural Head (Head 3): Attends broadly across earlier tokens, focusing on newline characters, punctuation, and speaker prefixes to maintain discourse context.

Head Count Ablation

Figure 10: Multi-Head Count Ablation on Tiny Shakespeare

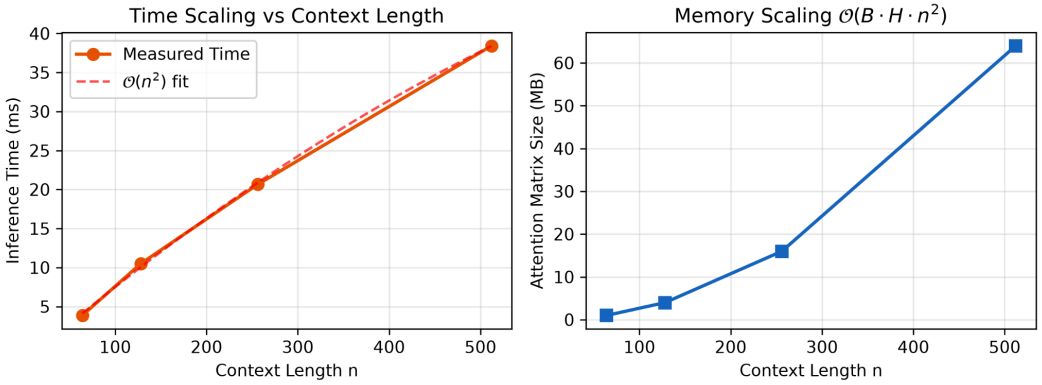


Keeping model dimension fixed at d=128:

- $H = 1\ (d_k = 128)$: 2.15 nats (best validation loss)
- $H = 4\ (d_k = 32)$: 2.22 nats
- $H = 8\ (d_k = 16)$: 2.23 nats
- For small embedding dimensions (d=128), splitting into 8 heads shrinks head dimension to only $d_k = 16$, limiting representational capacity per head. Fewer, larger heads performed best in this regime.

Computational Scaling

Figure 12: Context Length Scaling Benchmark



Context Length (n)	Forward Latency	Attention Matrix Memory
64	3.86 ms	1 MB
128	6.84 ms	4 MB
256	13.91 ms	16 MB
512	38.41 ms	64 MB

6. Analysis

1. Sequential vs. Parallel Computation:
    - Recurrent networks (RNN, LSTM, GRU) process timesteps sequentially (O(n) steps), preventing hardware parallelization across time.
    - Transformers parallelize all token interactions during training (O(1) sequential operations), achieving higher throughput even with much larger parameter counts.
2. Inductive Biases:
    - RNNs possess a hard-wired temporal bias (step-by-step decay), making them parameter-efficient for short or simple sequence problems.
    - Transformers have minimal temporal inductive bias and must learn relationships purely from data. However, direct pairwise attention eliminates gradient vanishing over distance, allowing Transformers to scale to much higher accuracy on both sensor classification (91.62% vs 87.37%) and language generation (3.16 BPC).